

第十届全国大学生 集成电路创新创业大赛

基于 RISC-V 指令集的 CPU 设计 后续优化说明报告

参赛杯赛：“竞业达”企业命题

队伍编号：CICC1004252

团队名称：和溢位

目录

1	报告说明	2
2	整体优化思路	2
3	主要优化内容	2
3.1	早期性能优化	2
3.2	基于西北赛区样例的后续优化	3
3.3	性能结果	6
3.4	西北赛区样例结果	6
3.5	性能与 WNS 走向	7
4	验证与取舍	8
5	总结	8
A	关键优化提交索引	9

1 报告说明

分赛区决赛主报告完成后，我们将主要精力转向处理器性能与 FPGA 时序优化。此时数字孪生平台已经关闭，后续工作主要通过 Verilator、Vivado XSim 和实现时序报告验证；由于数字孪生平台的关闭，缺乏上板验证的依据，而主报告内容则均经过正式上板验证，我们选择将后续优化单独写成本报告，作为对主报告的补充。

主报告所描述的设计基于较旧的版本，但架构没有改变。本报告将不再重复主报告中的完整架构等信息，主要补充说明此后完成的主要优化及量化结果。

优化始终保持**顺序单发射**的总体架构。我们通过性能计数器、周期级 RTL 仿真、Vivado 布局布线结果指导优化：先定位主要停顿来源，再用小范围改动验证假设，最后同时检查功能、周期以及建立/保持时序。

性能数据来自 Verilator 周期级 RTL 仿真，并按对应实现频率换算为评估时间。仅有仿真收益而无法达到时序的方案不作为正式结果。

本报告保留两套性能数据：`withMext-v2` 用于贯穿主报告版本至最终版本的统一 RTL 仿真复测；西北赛区样例用于呈现后期优化对更大程序及不同访存、分支特征的效果。前者每次完整运行均提交 380,344,384 条动态指令，表中时间由完整周期数除以 280 MHz 得到；后者约含 14.47 亿条动态指令。两套程序的运行时间**不能横向比较**，本报告只比较同一程序内部相邻版本的变化。

2 整体优化思路

1. **用数据定位瓶颈**。软件仿真快速统计、添加更多性能计数器。
2. **优先减少高频停顿**。主要针对 `load-use` 相关和乘法器。
3. **控制组合路径规模**。缩小旁路范围、减少宽 Mux、局部化地址生成和简化接口。
4. **固定项目实施**。对 RTL、IP 参数配置和 Vivado 项目统一纳入 Git 仓库管理。

3 主要优化内容

3.1 早期性能优化

主报告版本后，我们先围绕乘法、方向预测、缓存写入和 `load-use` 停顿开展优化：

- **缩短乘法流水延迟**。将乘法器从 6 周期逐步缩短并验证 5、4、3 周期方案。4 周期方案在性能与时序之间取得较好平衡；3 周期虽然更快，但产生大范围时序违例，因而没有作为最终方案。

支撑：性能计数器在完整程序中记录到约 1050 万次普通乘法，说明每减少一拍都

可能形成稳定收益。

- **改进分支方向预测。**为条件分支加入 2 bit 饱和计数器。该程序原有预测准确率已经较高，因此收益不大，但改动能够保持功能和时序正确。

支撑：withMext-v2 完整仿真记录的预测准确率约为 98.75%，与该项收益较小的现象一致；

- **优化 DCache 的整字写行为。**原设计在 store 后使缓存项失效，导致热点中的后续 load 反复未命中。新方案对可缓存的整字 sw 执行 write-allocate，同时用写入优先级和 epoch 防止较早的 load fill 覆盖新数据；byte/half store 仍采取保守失效策略。

支撑：计数器记录到约 2145 万次可缓存整字 store，且相邻版本时间从 1.943 s 降至 1.868 s。

- **优化 load-use 数据相关。**性能计数表明主要瓶颈不是分支，而是 load 后紧邻的 ADD/ADDI。晚数据旁路允许这类消费者提前进入 EXU，在缓存命中数据到达时直接替换源操作数；若数据尚未就绪则保持指令，避免使用无效数据。

支撑：完整计数器分别观察到 57,668,883 次 rs1 和 5,312,000 次 rs2 的相邻 load-ADD/ADDI 机会；对应仿真时间从 1.868 s 降至 1.643 s。

说明：我们把适用范围限制为容易验证的加法消费者，并将普通加法结果与重定向路径拆开，避免 DCache 命中信号经 ready 链一路影响 IFU，从而控制新增旁路的时序代价。

- **提高工程可复现性。**修复 Vivado 工程曾读取旧目录生成 RTL 的路径问题，统一项目内打包源、IP 输出物、时钟参数和实现策略，确保报告的物理结果确实对应当前候选。

3.2 基于西北赛区样例的后续优化

这一阶段的优化方向来自西北赛区样例的完整性能统计、NEMU 模型和固定 RTL 前缀。两套样例的汇总结果在后文分别展示。

- **练习 B 扩展功能。**尝试练习实现了一些 RISC-V B 扩展功能。较复杂的操作采用多周期执行与明确的 ready/valid 控制，在保证功能的同时限制单周期逻辑深度。
- **重新收敛 280 MHz 时序。**load-use 候选在 280 MHz 下仍有 WNS -0.084 ns 违例。我们把跨模块的宽控制改为粗粒度类别，在多周期单元内部完成具体译码；同时为 LSU 地址生成设置独立的 18 bit 通路，把旁路选择移到地址加法进位链之后，缩短“选择—加法—地址输出”的串联路径。

支撑：对应实现把 WNS 从 -0.084 ns 恢复到 0.000 ns；同一完整仿真的周期数保持不变，说明这轮调整解决的是组合路径，而不是以增加停顿换取时序。

说明：在 RTL 结构缩短后，再用 post-route 物理优化完成最后的布线整理。

- 改进乘除法实现。**低位乘法使用更短流水，乘法结果选择从通用结果 Mux 中局部化；除法采用 32 轮恢复除法，每拍完成一次移位、比较和减法，并用明确的 ready/valid 握手控制流水停顿。最后一轮的符号修正后来单独分拍，避免 33 bit 减法与 32 bit 取负串在同一关键路径上。
支撑：M 扩展定向测试全部通过；符号修正分拍在固定前缀中只增加 12 周期，并移除了原除法结束路径对应的时序违例族。
- 提升窄加载和缓存局部性。**这里的 DCache shadow 是主缓存状态的一份轻量快速副本，保存快速命中判断所需的 valid、tag 和数据字节；它不是新的一级或二级缓存。设计保证 shadow 与主缓存一致：load fill 时同步更新，store 命中时按写掩码更新，无法安全合并时保守失效。
支撑：西北赛区样例的完整模型找到 87,172,384 个可节省周期，并指出热点集中在约 7.36 亿条指令之后，短前缀不足以代替完整运行。
说明：在 shadow 上直接完成 LBU/LHU 的字节、半字提取，可让紧邻的简单消费者更早获得正确数据，同时避免旧数据被错误旁路。
- 保留部分写命中的缓存行。**对能够安全合并的部分 store，不再像旧设计那样使整条 shadow 缓存行失效，而是按照 byte-enable 只更新被写入的字节并保留其余字节。这样既维持了 shadow 与主存写入的一致性，又减少了随后读取同一缓存行时重新 miss 的概率。
支撑：完整缓存模型的总命中率由 86.0341% 提高到 95.8535%，预测节省 298.394 ms；实际记录由 6.481 s 降至 6.182 s，差值为 299 ms。
- 扩大并优化 BTB。**将 BTB 的有效物理项扩展为 32 项，并把更新 shadow 放入 LUTRAM，减少容量冲突。主查询存储保留预测目标等完整表项；较窄的更新 shadow 仅保存 valid、tag、分支类型和 2 bit 方向计数器，使更新端无需绕回主查询数据通路。将它改为 LUTRAM 后，预测语义不变，但减少触发器、时钟扇出和布线压力。
支撑：模型预计 32 项 BTB 少 4,187,905 次 miss；RTL A/B 标定每次 miss 为 3 周期，换算 44.870 ms，与 6.553 s 到 6.508 s 的 45 ms 变化一致。LUTRAM 化前后固定前缀周期完全相同。
- 识别并预测标准返回。**RISC-V 的标准 ret 是伪指令，其精确机器码对应 jalr x0, 0(x1)，即 0x00008067。译码级直接比较这一完整指令编码：只有完全匹配时，才把 BTB 中已记录的目标作为无条件预测目标；其他 JALR 仍由原执行级路径解析。这样不需要增加通用 JALR 的 32 bit 目标比较器，也不会改变非标准间接跳转的执行语义。
支撑：模型预计减少 2,575,420 次 miss，即 27.594 ms；记录由 6.508 s 降至 6.481 s。
- 缩短关键接口并分拍除法修正。**在部分写命中版本之后，我们继续缩短返回地址

旁路：当 `ret` 依赖 EXU 正在产生的 `x1` 时，不再要求同拍完成预测目标比较，而是保守地交给原重定向路径处理；同时删除重复地址字段，将跨模块数据缩窄到实际使用的 22 bit。随后把同拍 ALU 旁路从缓存/LSU/WBU 的通用选择树中分离，把 WBU 到地址生成器的专用旁路缩减为 22 bit 数据和一个有效位，并把 DCache 查询接口从完整 32 bit 地址缩减为实际需要的 index 和 tag。这些接口调整在本次完整仿真中均未增加周期。

这一项中唯一有明确周期代价的是迭代除法器分拍。原先最后一轮迭代需要在同一拍完成 33 bit 减法 and 32 bit 符号修正；新设计增加 `correct` 状态，先寄存无符号商和余数，下一拍再做可选的二进制补码修正。因此，每次普通除法或求余多占用一拍。本次完整仿真中共有 12 次此类操作，因此总周期精确增加 12，即在 280 MHz 下约 $0.043\ \mu\text{s}$ 。其余接口缩减均未改变周期数。上述 RTL 修改将违例缩小到约 $-0.004\ \text{ns}$ 。

- **选择匹配网表的实现策略。**在完成上述 RTL 结构调整后，再使用 Auto/Explore、NoTimingRelaxation 和 post-route 物理优化关闭最后几皮秒的布线违例，最终得到 WNS +0.013 ns。因此最终无违例结果来自“RTL 关键路径缩减、除法修正分拍、实现策略选择”的组合，而不是单独更换实现策略；这也解释了时序重新收敛时出现的 12 周期代价。

3.3 性能结果

表 1: 统一采用 withMext-v2 的完整 RTL 仿真性能演进

优化里程碑	频率	仿真时间	较前基线提升	时序状态
主报告版本	280 MHz	2.020 s	—	WNS +0.050 ns
乘法与方向预测优化	280 MHz	1.943 s	3.8%	WNS +0.031 ns
DCache 整字写分配	280 MHz	1.868 s	3.9%	WNS +0.019 ns
load-use 晚数据旁路	280 MHz	1.643 s	12.0%	WNS −0.084 ns
迭代 RTL 除法器	280 MHz	1.606910 s	2.2%	通过
LSU 地址通路与 280 MHz 收敛	280 MHz	1.606910 s	0.0%	WNS 0.000 ns
窄加载与 DCache shadow	280 MHz	1.607404 s	−0.031%	WNS 0.000 ns
32 项 BTB 与更新 shadow	280 MHz	1.607403 s	+0.000055%	WNS −0.011 ns
精确 ret 识别与预测	280 MHz	1.607403 s	+0.000013%	WNS −0.091 ns
部分写命中保留	280 MHz	1.607403 s	+0.000001%	WNS −0.072 ns
接口缩减与除法修正分拍	280 MHz	1.607403 s	−0.000003%	WNS −0.004 ns
最终时序闭合版本	280 MHz	1.607403 s	0.0%	WNS +0.013 ns

最终版本完整运行 450,072,868 周期、380,344,384 条指令，CPI 为 1.183330。相对主报告版本的 2.020 s，统一口径下仿真时间缩短约 **20.4%**。后期若干优化主要用于减少资源与关闭时序，在该自测程序中的周期变化很小；表中仍保留精确百分比，避免把时序优化误写成性能收益。最终实现的建立时间 WNS 为 +0.013 ns、保持时间 WHS 为 +0.072 ns。

3.4 西北赛区样例结果

西北赛区样例与 withMext-v2 的动态指令组成不同，以下数据仅在本表内部逐行比较。表中里程碑与前述优化项一一对应；LUTRAM shadow、返回旁路和接口缩减主要改善时序，周期不变或只有已量化的小代价。

表 2: 西北赛区样例的性能与时序演进

优化里程碑	评估时间	较前基线提升	时序状态
多周期单元与 280 MHz 收敛	6.866 s	—	WNS 0.000 ns
窄加载与 DCache shadow	6.553 s	4.6%	WNS 0.000 ns
32 项 BTB 与更新 shadow	6.508 s	0.7%	WNS -0.011 ns
精确 ret 识别与预测	6.481 s	0.4%	WNS -0.091 ns
部分写命中保留	6.182 s	4.6%	WNS -0.072 ns
接口整理与最终时序闭合	6.184 s	-0.03%	WNS +0.013 ns

最终结果相对 6.866 s 基线缩短约 **9.9%**。6.182 s 到 6.184 s 的小幅回退出现在除法修正分拍和接口整理之后；固定前缀明确观察到 12 次除法各增加一拍，而最终版本同时将 WNS 从负值收敛到 +0.013 ns，因此选择 6.184 s 的时序闭合版本作为最终结果。

3.5 性能与 WNS 走向

图中蓝线为同一样例内部的评估时间，红色虚线为对应实现的 WNS；灰色零线以上表示建立时序通过。纵轴分别缩放，折线用于观察趋势，不用于比较两套样例的绝对时间。

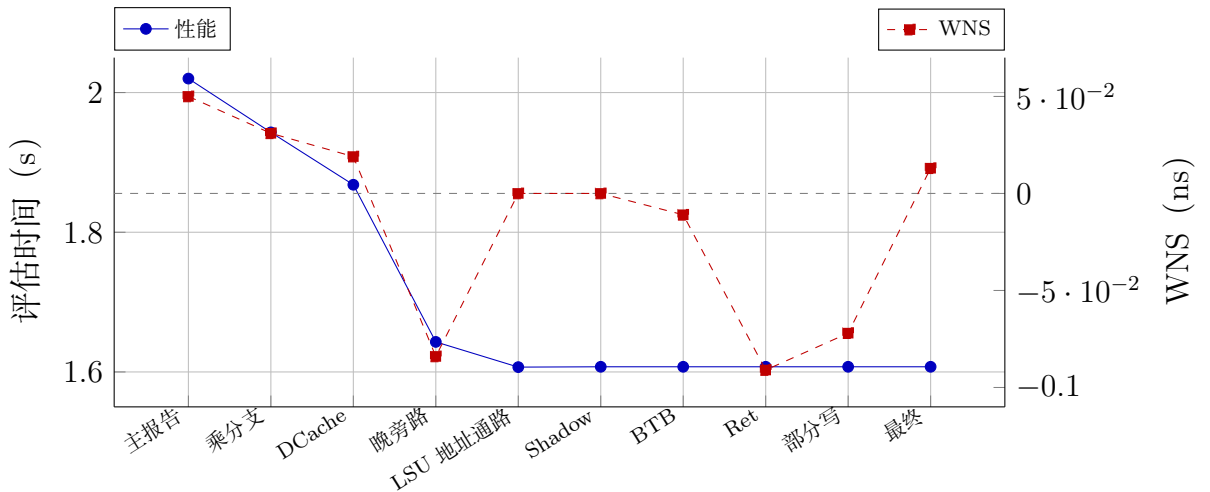


图 1: withMext-v2 的性能与 WNS 走向

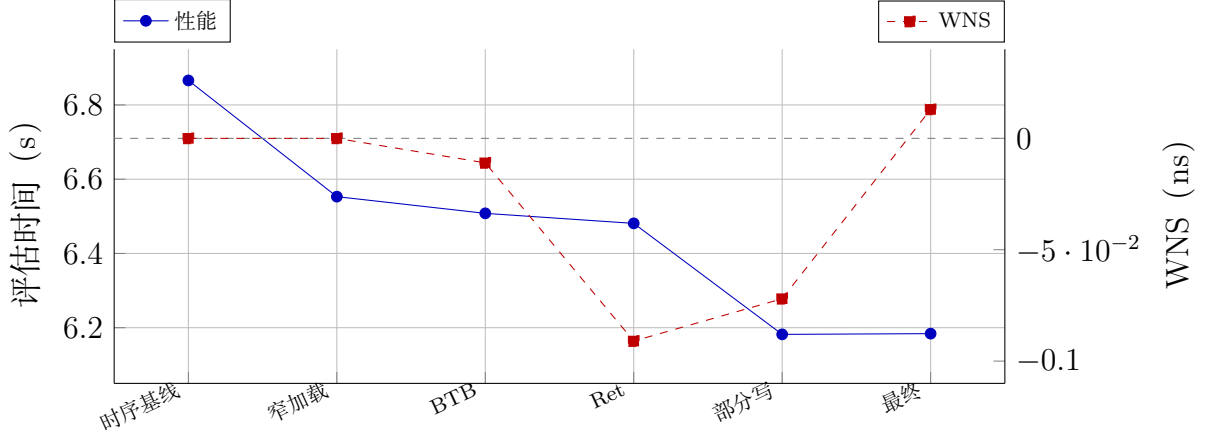


图 2: 西北赛区样例的性能与 WNS 走向

4 验证与取舍

优化过程中同时保留成功和失败实验，避免只依据最好的一次数据选择方案。功能验证覆盖基础 CPU 测试、load-store、分支与递归测试、M 扩展定向架构测试，以及相关功能练习测试；涉及 CSR 的候选还运行 RT-Thread 至命令行提示符。性能候选使用完整程序或固定代表性前缀核对周期趋势，并通过 Vivado 实现结果检查目标频率下的建立与保持时序。

若某方案明显减少周期但产生大范围负 WNS，例如把 DCache 查询前移到 IDU、使用宽通用队列、过度扩展组合旁路或采用 3 周期乘法器，我们会保留实验结论但回退实现。轻微负 WNS 的候选用于分析优化方向和呈现演进过程，不作为最终闭合版本。最终版本还要求 PLL 请求频率与实际生成频率一致，且 setup/hold 均通过。

5 总结

在不改变顺序单发射总体架构的前提下，后续优化把工作重点从“完成功能”推进到“依据真实热点降低周期，并让优化后的网表稳定通过 FPGA 时序”。统一采用 withMext-v2 完整仿真后，280 MHz 换算时间由主报告版本的 2.020 s 降低到最终版本的 1.607403 s，缩短约 20.4%；西北赛区样例在自身统一口径下由 6.866 s 降低到最终时序闭合版本的 6.184 s，缩短约 9.9%。两组数据相互独立，但共同说明优化方向在不同程序特征下具有可观察效果。

这些工作不仅包含乘法、缓存、旁路、分支预测和相关功能练习，也包括 IP 接口核对、RTL 除法器、Vivado 输入固定和实现策略选择。最终结果表明，处理器在功能正确的基础上进一步提高了有效吞吐率，并保持了可复现的实现时序。

A 关键优化提交索引

下表给出正文关键里程碑对应的完整 Git commit hash。部分里程碑由多个小提交共同构成；表中按功能边界列出，便于结合提交说明和 RTL diff 复核。

表 3: 关键优化与 Git 提交对应关系

关键内容	完整 commit hash
主报告性能基线	d41aa7495b7b5a19f666ad627f998debb232b5b0
乘法、方向预测与整字写分配组合	196077bdd8187c62e6e9988820ce27fffc87247e
load-use 晚数据旁路基线	d8364daf1df0ea203ab661bf5c093e208b9414f9
迭代 RTL 除法器	85d3dad160c1dfcc7ef200fc14da22b76319dc77
LSU 地址旁路后移	11f93f5f4cb41e637056fc7b341461bc13a11af2
缓存窄加载 shadow	e71bb475819cfcec19bde6553f54ab37091db591
窄 store 的 shadow 一致性修复	5a73acc1b28c991076a1804d434c8a9e4448ad63
热点 LBU/LHU 提取通路	b5731d80617cf29ff2d8a76d71f8c64c6dca0443
多周期单元内部精确译码	d71fa1a364500fe00c27440e821ee80d92d0b743
有符号除法溢出迭代处理	906249b9443b2b25ee1e064f9f3ab296d75d9eea
启用 32 项 BTB	8eb6b8ccd5538f099765e664b0bab63013c483a8
BTB 更新 shadow 改用 LUTRAM	a72b1f9c2756ccf211eb8bb897b06b1acb3dbf7f
仅对精确 ret 使用间接预测	f5143a0c398e9f4cb0f2b3c1e7d34b286b5a179b
保留部分写命中的缓存行	d205c9aa7a41c8d8611033fd6aee0573fb863014
缩短返回地址旁路	3fd8030b5e0ef0028e31df3fcf3dd0737f090eba
缩短 ALU 旁路并分拍除法修正	c4279e438726cceae21fc71321ee5ee306bb95e9
缩窄 WBU 地址旁路	e3a7baaffdf632a1bddd790ae8cedb7df9fcf9cd
缩窄 DCache 地址接口	9343d53069f6bb268dd1210ae2b083a70699008f
最终冻结 RTL	58f371d5dbff5e6a8c730ab59ea6d39048012159
最终 Vivado 输入恢复	c6e15a9d0e43099bea863c45e5f0020a5bac597a